



Comprehensive Class Notes: Combining Session 6 (Creating Tables) and Session 7 (Azure SQL)

Overview

This set of notes combines key concepts from **Session 6: Creating Tables** and **Session 7: Azure SQL** from the "Intelligent Data Management with SQL Server" course. The focus is on creating and managing tables in SQL Server (on-premises) and understanding how to work with Azure SQL for cloud-based database management. The notes include detailed explanations, practical examples, classwork exercises, and tests to reinforce learning.

Session 6: Creating Tables

Key Objectives

- Understand how to create, modify, and delete tables in SQL Server using Transact-SQL (T-SQL).
- Apply constraints to ensure data integrity (e.g., PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT).
- Practice table creation and management with real-world scenarios.

Core Concepts

1. Table Creation:

- Use the `CREATE TABLE` statement to define a table with columns, data types, and constraints.
- Syntax:

```
CREATE TABLE <table_name> (  
    column_name1 data_type [constraint],  
    column_name2 data_type [constraint],  
    ...  
);
```

- Example:

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    ContactNumber VARCHAR(15),  
    Email VARCHAR(100) UNIQUE  
);
```

2. Constraints:

- **PRIMARY KEY:** Ensures unique and non-null values for a column.
- **FOREIGN KEY:** Links two tables to enforce referential integrity.
- **UNIQUE:** Ensures no duplicate values in a column (allows NULL).
- **CHECK:** Enforces a condition on column values.
- **DEFAULT:** Sets a default value if none is provided.
- Example:

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE DEFAULT GETDATE(),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

3. Modifying Tables:

- Use `ALTER TABLE` to add, modify, or drop columns/constraints.
- Example:

```
ALTER TABLE Customers
  ADD Address VARCHAR(200);
ALTER TABLE Customers
  DROP COLUMN Email;
```

4. Deleting Tables:

- Use `DROP TABLE` to remove a table and its data.
- Example:

```
DROP TABLE Customers;
```

5. Best Practices:

- Use meaningful column names and appropriate data types.
 - Apply constraints to enforce data integrity.
 - Test insertions to verify constraints.
-

Session 7: Azure SQL

Key Objectives

- Understand Azure SQL as a cloud-based relational database service.
- Learn how to create and manage databases in Azure SQL using SQL Server Management Studio (SSMS).
- Explore Azure SQL architecture, features, and benefits.

Core Concepts

1. Azure SQL Overview:

- Azure SQL is a cloud-based relational database service based on SQL Server, offering scalability, high availability, and managed infrastructure.
- Key services: Azure SQL Database, Azure SQL Managed Instance, and SQL Server on Azure VMs.

2. Azure SQL Architecture:

- Comprises compute, storage, and networking layers managed by Microsoft Azure.

- Supports elastic scaling and automatic backups.

3. Features and Benefits:

- **Scalability:** Adjust resources dynamically (DTU or vCore models).
- **High Availability:** Built-in redundancy and failover.
- **Security:** Features like Transparent Data Encryption (TDE) and Advanced Threat Protection.
- **Compatibility:** Supports most SQL Server features, with some limitations (e.g., no `USE` statement for switching databases).

4. Creating an Azure SQL Database:

- Steps in Azure Portal:
 - a. Log in to Azure Portal.
 - b. Navigate to "Create a resource" > "SQL Database."
 - c. Configure database name, server, resource group, and compute tier.
 - d. Set authentication (SQL Server Authentication or Azure AD).
- Example T-SQL to create a table in Azure SQL:

```
CREATE TABLE Products (  
    ProductNo INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Description VARCHAR(255),  
    Price DECIMAL(10,2) CHECK (Price > 0)  
);
```

5. Connecting to Azure SQL:

- Use SSMS to connect to Azure SQL Database.
- Steps:
 - a. Open SSMS and select "SQL Server Authentication."
 - b. Enter the server name (e.g., `<server>.database.windows.net`), login, and password.
 - c. Right-click "Tables" in Object Explorer to create or manage tables.

6. Differences from On-Premises SQL Server:

- No `USE` statement for switching databases in Azure SQL Database.
 - Limited support for cross-database queries.
 - Managed infrastructure reduces administrative overhead.
-

Classwork Examples

Example 1: Creating a Table in SQL Server (On-Premises)

Scenario: A retail company needs a table to store product information.

```
CREATE TABLE RetailProducts (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100) NOT NULL,  
    Category VARCHAR(50),  
    Price DECIMAL(10,2) CHECK (Price >= 0),  
    Stock INT DEFAULT 0  
);  
  
-- Insert sample data  
INSERT INTO RetailProducts (ProductID, ProductName, Category, Price, Stock)  
VALUES (1, 'Laptop', 'Electronics', 999.99, 50);
```

Example 2: Creating a Table in Azure SQL

Scenario: The same retail company wants to store product data in an Azure SQL Database.

```
CREATE TABLE AzureRetailProducts (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100) NOT NULL,  
    Category VARCHAR(50),  
    Price DECIMAL(10,2) CHECK (Price >= 0),  
    Stock INT DEFAULT 0  
);  
  
-- Insert sample data  
INSERT INTO AzureRetailProducts (ProductID, ProductName, Category, Price, Stock)  
VALUES (1, 'Smartphone', 'Electronics', 799.99, 100);
```

Example 3: Adding a Foreign Key

Scenario: Create an `Orders` table referencing the `RetailProducts` table.

```
CREATE TABLE RetailOrders (  
    OrderID INT PRIMARY KEY,  
    ProductID INT,  
    Quantity INT CHECK (Quantity > 0),  
    OrderDate DATE DEFAULT GETDATE(),  
    FOREIGN KEY (ProductID) REFERENCES RetailProducts(ProductID)  
);  
-- Insert sample data  
INSERT INTO RetailOrders (OrderID, ProductID, Quantity)  
VALUES (101, 1, 5);
```

Example 4: Modifying a Table

Scenario: Add a `Discount` column to the `RetailProducts` table.

```
ALTER TABLE RetailProducts  
    ADD Discount DECIMAL(5,2) DEFAULT 0.00;
```

Classwork Exercises

Exercise 1: Table Creation in SQL Server

Task: Create a database for a library system with two tables:

- `Books` (BookID, Title, Author, ISBN, Price, AvailableCopies)
- `Borrowers` (BorrowerID, FirstName, LastName, Email, BorrowedBookID)

1. Define appropriate data types and constraints.
2. Insert at least three records into each table.
3. Write a query to retrieve all books borrowed by a specific borrower.

Sample Solution:

```

-- Create Books table
CREATE TABLE Books (
    BookID INT PRIMARY KEY,
    Title VARCHAR(100) NOT NULL,
    Author VARCHAR(50) NOT NULL,
    ISBN VARCHAR(13) UNIQUE,
    Price DECIMAL(10,2) CHECK (Price > 0),
    AvailableCopies INT DEFAULT 0
);

-- Create Borrowers table
CREATE TABLE Borrowers (
    BorrowerID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    BorrowedBookID INT,
    FOREIGN KEY (BorrowedBookID) REFERENCES Books(BookID)
);

-- Insert data into Books
INSERT INTO Books (BookID, Title, Author, ISBN, Price, AvailableCopies)
VALUES
    (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '9780743273565', 12.99, 10),
    (2, '1984', 'George Orwell', '9780451524935', 9.99, 15),
    (3, 'To Kill a Mockingbird', 'Harper Lee', '9780446310789', 14.99, 8);

-- Insert data into Borrowers
INSERT INTO Borrowers (BorrowerID, FirstName, LastName, Email, BorrowedBookID)
VALUES
    (101, 'Alice', 'Smith', 'alice.smith@example.com', 1),
    (102, 'Bob', 'Johnson', 'bob.johnson@example.com', 2),
    (103, 'Carol', 'Williams', 'carol.williams@example.com', 3);

-- Query to retrieve books borrowed by BorrowerID 101
SELECT b.Title, b.Author, br.FirstName, br.LastName
FROM Books b
JOIN Borrowers br ON b.BookID = br.BorrowedBookID
WHERE br.BorrowerID = 101;

```

Exercise 2: Table Creation in Azure SQL

Task: Create an Azure SQL Database named `LibraryDB` and replicate the `Books` and `Borrowers` tables from Exercise 1. Use SSMS to connect and execute the T-SQL code. Insert the same sample data and run the same query.

Steps:

1. In Azure Portal, create a new SQL Database named `LibraryDB`.
 2. Connect to the database using SSMS with the server name and credentials.
 3. Execute the T-SQL code from Exercise 1 to create tables and insert data.
 4. Run the query to verify the results.
-

Tests

Test 1: Table Creation and Constraints (SQL Server)

Question: Create a table `Employees` with the following specifications:

- Columns: `EmployeeID` (INT, PRIMARY KEY), `FirstName` (VARCHAR, NOT NULL), `LastName` (VARCHAR, NOT NULL), `DepartmentID` (INT, FOREIGN KEY), `Salary` (DECIMAL, CHECK > 0).
- Create a `Departments` table with `DepartmentID` (INT, PRIMARY KEY) and `DepartmentName` (VARCHAR, NOT NULL).
- Insert 3 records into each table.
- Write a query to list employees in a specific department.

Sample Solution:


```

-- Create Departments table
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL
);

-- Create Employees table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    DepartmentID INT,
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);

-- Insert data into Departments
INSERT INTO Departments (DepartmentID, DepartmentName)
VALUES
    (1, 'HR'),
    (2, 'IT'),
    (3, 'Sales');

-- Insert data into Employees
INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary)
VALUES
    (101, 'John', 'Doe', 1, 50000.00),
    (102, 'Jane', 'Smith', 2, 60000.00),
    (103, 'Bob', 'Johnson', 2, 55000.00);

-- Query to list employees in IT department
SELECT e.FirstName, e.LastName, e.Salary, d.DepartmentName
FROM Employees e
JOIN Departments d ON e.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'IT';

```

Test 2: Azure SQL Database Creation

Question: Create an Azure SQL Database named `EmployeeDB` and replicate the `Departments`

and `Employees` tables from Test 1. Insert the same data and run the query to list employees in the IT department.

Steps:

1. In Azure Portal, create a new SQL Database named `EmployeeDB`.
2. Connect to `EmployeeDB` using SSMS.
3. Execute the T-SQL code from Test 1.
4. Verify the query results.

Test 3: Constraint Violation Handling

Question: Using the `Employees` table from Test 1, attempt to insert a record that violates:

1. The PRIMARY KEY constraint.
2. The FOREIGN KEY constraint.
3. The CHECK constraint.

Write the error messages you expect to see.

Sample Solution:

```
-- Attempt to violate PRIMARY KEY (duplicate EmployeeID)
INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary)
VALUES (101, 'Alice', 'Brown', 1, 52000.00);
-- Expected Error: Violation of PRIMARY KEY constraint. Cannot insert duplicate key in object 'dbo

-- Attempt to violate FOREIGN KEY (non-existent DepartmentID)
INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary)
VALUES (104, 'Alice', 'Brown', 999, 52000.00);
-- Expected Error: The INSERT statement conflicted with the FOREIGN KEY constraint.

-- Attempt to violate CHECK (negative Salary)
INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary)
VALUES (104, 'Alice', 'Brown', 1, -100.00);
-- Expected Error: The INSERT statement conflicted with the CHECK constraint.
```

Key Takeaways

- **Session 6:** Tables are the foundation of SQL Server databases. Use `CREATE TABLE`, `ALTER TABLE`, and `DROP TABLE` to manage tables, and apply constraints to ensure data integrity.
 - **Session 7:** Azure SQL provides a scalable, cloud-based alternative to on-premises SQL Server. It supports similar T-SQL syntax but requires configuration through the Azure Portal and SSMS for management.
 - **Practical Application:** Both on-premises and Azure SQL databases require careful planning of table structures, constraints, and data integrity rules to meet business needs.
-

Additional Notes

- **Testing Environment:** Use SQL Server Management Studio (SSMS) with the AdventureWorks2019 sample database for on-premises exercises. For Azure SQL, set up a free Azure account or use a sandbox environment.
 - **Best Practices:**
 - Always back up databases before dropping tables or making significant changes.
 - Test constraints by attempting invalid inserts to understand error handling.
 - In Azure SQL, ensure proper firewall rules and authentication settings for secure access.
 - **Further Practice:** Explore the AdventureWorks2019 database for complex table relationships and try deploying similar structures in Azure SQL.
-

End of Combined Notes for Sessions 6 and 7

For additional practice, create more complex table relationships and test them in both SQL Server and Azure SQL environments. Use the "Try It Yourself" sections from the original document for further exercises.